Caching in LLMs — Project Overview

Four 10-minute reads that cover the whole project

Nissim Brami
nissimbrami@post.bgu.ac.il
Ben-Gurion University of the Negev • Prof. Gil Einziger

This document is the "explain-me-the-whole-project" single page. It is written for someone who has **not read the paper** and has **not used GPTCache**. Read the four parts below in order. Each takes about 10 minutes at a normal reading pace. If you can hold these four sections in your head, you can explain the whole project to anyone.

Full detail lives in the rest of the repository:

- Task 1 (StreamingLLM presentation): `task1-presentation/`
- Task 2 (GDSF cost-aware eviction for GPTCache): `task2-final-project/`

1 What the StreamingLLM paper is (10 minutes)

1.1 The problem, in one sentence

Large language models cannot **keep talking to you** for hours on end without either running out of memory or producing gibberish. The StreamingLLM paper explains why, and fixes it in about 50 lines of code.

1.2 The setting

When you type into ChatGPT or a similar system, the model generates one token at a time. To generate token $T+1$, it has to look at every token from 1 through T. Recomputing that lookup from scratch every step would be quadratic ($O(T^2)$), so the model keeps a **KV cache** — a list of key/value vectors, one per past token, sitting in GPU memory. This is the object the paper is about.

Two ugly facts about the KV cache:

1. It **grows linearly** with the length of the conversation. A 7-billion-parameter model at 4,000 tokens uses ∼2 GB of KV cache in FP16. Long conversations eventually don't fit in GPU memory.
2. Language models were only pre-trained on some fixed window (say 4,000 tokens). If the cache holds more, the positional encodings inside the model go out of distribution and quality collapses.

So there is a hard cap on how long a conversation can be. Once you hit it, you have three obvious options — and **all three are broken**.

1.3 The three broken baselines

Llama-2-13B, first book of PG19, 65,000 tokens. Perplexity, lower is better.

Baseline	Cost	Perplexity
Dense attention (keep all)	$O(T^2)$	5641 (broken, then OOM)
Window attention (keep last L)	$O(TL)$	**5158** — gibberish
Window + recompute (rebuild every step)	$O(TL^2)$	5.43 (correct, too slow)

The mystery is line 2. Keeping only the most recent tokens should be the cheapest reasonable strategy. But the model completely falls apart — from a fluent perplexity of 5-ish to 5158. Why?

1.4 The observation — attention sinks

When the authors visualized where a language model looks, they saw something odd: **starting from layer 3 onward, every attention head in every layer of every model tested (Llama-2, MPT, Falcon, Pythia) heavily attends to the very first few tokens** — regardless of what those tokens actually say. They named these initial tokens **attention sinks**.

Why? Because the softmax function that produces attention weights **must sum to one**. The model always has a fixed budget of attention mass to spend at every step, whether the context deserves it or not. That excess mass has to land somewhere. And in a decoder-only model, the tokens visible to *every* subsequent position are the initial ones. They become the natural dumping ground.

The moment you evict the first token from the cache, you rip a huge chunk out of the softmax denominator, and every remaining attention weight in the row gets renormalized. The whole attention distribution warps. Perplexity $\rightarrow$ 5158.

1.5 The clincher experiment

Is it the content of the first tokens that matters, or just their *position*? The cleanest ablation in the paper: replace the first four tokens with linebreak characters (`\n`). Nothing else changes.

Configuration	Perplexity
Window, no sinks	5158.07
StreamingLLM (4 real tokens + 1020 rolling)	**5.40**
StreamingLLM with linebreaks at positions 0–3	5.60

Almost identical. **The tokens' positions do the work, not their content.** Sinks are structural, not semantic.

1.6 The fix (StreamingLLM)

Trivial once you've seen the mechanism:

1. Always keep the **first 4 tokens** in the cache. Never evict.
2. Also keep the **last L tokens** in a rolling window (FIFO).
3. Everything between the sinks and the rolling window is discarded.
4. **Renumber position IDs inside the cache**, not in the original text. If the cache holds tokens with original positions [0,1,2,3,6,7,8] and we're decoding position 9, the model sees positions [0,1,2,3,4,5,6,7]. Sinks and window become adjacent in position space.

No fine-tuning. No new model architecture. About 50 lines of PyTorch on top of a Hugging Face model.

1.7 The results

- **Perplexity flat past 4,000,000 tokens** on ten models: Llama-2 {7B, 13B, 70B}, MPT {7B, 30B}, Falcon {7B, 40B}, Pythia {2.8B, 6.9B, 12B}.
- **Up to 22.2× faster** per token than the sliding-window-with-recomputation baseline.
- Streaming QA (concatenated ARC) matches or slightly beats the one-shot-per-question oracle. Meanwhile window attention collapses to 0.12% on 70B.

1.8 The limitations

Honest about them: it does **not** extend the context window, it underperforms simple truncation on long-document QA (LongBench), bigger cache does not monotonically lower perplexity, and it does not compare to content-aware evictors (H2O, SnapKV). Adopted upstream within months: NVIDIA TensorRT-LLM, HF Transformers, Intel Extension for Transformers, MLC LLM.

One-line summary: StreamingLLM is a small, robust, honest empirical fix for a softmax quirk, and it is exactly the kind of paper a caching class should present because it *is* a cache eviction policy.

2 What we did for Task 1 (10 minutes)

2.1 The assignment

Prof. Einziger's Task 1: pick a recent caching-in-LLMs paper, present it to the class in a ∼60-minute lecture. Deliverables: PowerPoint deck + speaker notes + optional supporting materials.

2.2 The paper we chose

Efficient Streaming Language Models with Attention Sinks by Xiao, Tian, Chen, Han, Lewis (ICLR 2024). The paper summarized in §1 above. Why this paper for a caching audience:

- It is *literally* a paper about a cache eviction policy — the KV cache in a language model.
- It uses course vocabulary (capacity, eviction, hit, miss) without ever using the word "cache".
- It has a clean empirical result and an honest limits section, which makes for a good 60-minute talk.
- It connects cleanly to Task 2 — both projects are eviction policies on bounded caches, at different layers of the stack.

2.3 What we built

A single directory `task1-presentation/` with:

- **The paper itself** — `papers/streaming-llm/` + `deep-read.md` (page-by-page summary) + `critical-analysis.md` (our critique).
- **The deck** — `StreamingLLM_Presentation.pptx`, **46 slides**, built programmatically by `build_streaming_llm_deck.py` so every slide is version-controlled Python.
- **Speaker notes** — `notes/speaker-notes.md`, ∼8,000 words, one section per slide, every numeric claim tied to a page.

- **Runnable demo** — `code/streaming_llm_demo.py`, ∼200 lines of PyTorch + Hugging Face, with `SinkKVCache`/`WindowKVCache`/`DenseKVCache` classes side by side + unit tests.
- **Reference decks** — four prior course decks kept as style references.

2.4 How the deck is structured (60 minutes)

Twelve sections: Basics (5 min) → Related work (5 min) → Observation (6 min) → Mechanism (10 min) → Sink-token pre-training (4 min) → Results (12 min) → Limits deep-dive (6 min) → Modern alternatives (3 min) → Implementation walkthrough (4 min) → What I would change (2 min) → Bridge to Task 2 (1 min) → Q&A prep.

2.5 The elevator pitch

> *A 2024 ICLR paper explaining why language models fall apart on long conversations, and a small trick — always keep the first 4 tokens in the cache plus a rolling window of the last L — that fixes it. It's a cache eviction policy, which is why we picked it for a caching class.*

3 What GPTCache is (10 minutes)

3.1 The problem, in one sentence

Calling GPT-4 twice with **semantically equivalent** prompts costs twice as much and takes twice as long — even though the model would give roughly the same answer. GPTCache puts a **response cache** in front of the LLM so that "similar enough" prompts return the cached answer instead of hitting the paid API.

3.2 Where it lives

- Repository: `zilliztech/GPTCache` on GitHub. Company: Zilliz (the people behind the Milvus vector database).
- License: **Apache 2.0**.
- Stars: ∼**12,000**.
- First release: mid-2023, riding the wave of production LLM apps.
- Language: Python.
- Position in the stack: sits *between your application and the OpenAI/Anthropic/Hugging Face API.*

3.3 How the caching works

Not a normal key-value cache — prompts are almost never exactly equal. Two things need to match:

1. **The prompt has to be semantically similar** to a previously cached prompt. GPTCache embeds the prompt into a dense vector (via SentenceTransformers, OpenAI embeddings, etc.), and looks up the nearest neighbours in a vector database (FAISS, Milvus, ChromaDB, etc.).
2. **The cached answer has to still be valid.** Every entry has metadata (model, temperature, system prompt) and only matches with matching metadata are used.

Three components you interact with:

- `CacheBase` — where prompts and answers are stored (SQLite, PostgreSQL, MongoDB...).
- `VectorBase` — where the embeddings are stored (FAISS, Milvus, Chroma...).
- `EvictionBase` — the eviction policy for when the caches get full. **This is where our contribution goes.**

3.4 The eviction question

Before our change, `EvictionBase(name="memory", policy=P)` accepted four options for P:

- **LRU** — evict whichever entry hasn't been touched in the longest time. (Default.)
- **LFU** — evict whichever entry has the smallest hit count.
- **FIFO** — evict in insertion order.
- **RR** — evict a uniformly random entry.

These are the canonical page-cache policies from an operating systems textbook. They all share a problem for LLM traffic: they treat every cached entry as equally valuable. But regenerating one entry might cost 20 cents (a long GPT-4 completion) and regenerating another might cost 0.02 cents (a short GPT-3.5 answer). LRU has no way to know that. So on realistic LLM traffic, these policies **evict expensive answers as readily as cheap ones**, and every avoidable eviction of an expensive answer costs real money.

3.5 What GPTCache already did well, and what was missing

Already excellent: backend flexibility (swap SQLite for MongoDB transparently), similarity strategies (exact match, kNN, distance thresholds), and integrations with LangChain / LlamaIndex / OpenAI SDK.

Missing: no cost-aware or size-aware eviction policy. Every option treats entries as fungible.

Because `EvictionBase` is already a factory with a clean `policy=...` argument, adding a new policy is exactly the kind of contribution maintainers welcome. That's the surface where our GDSF policy plugs in.

4 What we did for Task 2 (10 minutes)

4.1 The assignment

An open-source contribution + a research paper on a caching-in-LLMs topic. Deliverables: a real PR-ready contribution to an upstream repo, plus an 8-page ACM-`acmart` sigconf paper.

4.2 What we chose

Add a new eviction policy to GPTCache: **GDSF** — **Greedy-Dual-Size-Frequency**. Written by Cao & Irani (USITS 1997) and Cherkasova (HP Labs 1998) for **web proxy caches** in the late 1990s. A weighted-caching classic that has never been ported to LLM caches.

4.3 The GDSF idea, in one line

Assign each entry a **priority score** that folds in three signals:

$$\text{Priority}(i) \;=\; L \;+\; \frac{\text{freq}(i)^{\alpha} \cdot \text{cost}(i)^{\beta}}{\text{size}(i)}$$

- $\text{freq}(i)$ — request count. Rewards hot entries.
- $\text{cost}(i)$ — generation cost. Rewards expensive answers you don't want to regenerate.
- $\text{size}(i)$ — storage size. Penalizes bloat.
- L — a monotone "clock" that inflates on every eviction, so recent entries stay competitive even before their frequency rises. This is what keeps GDSF from getting stuck.
- α, β — exponents that tune frequency vs. cost.

Evict the entry with the **lowest priority**. Cao & Irani proved this is competitively optimal (ratio k) for weighted online caching among deterministic algorithms.

4.4 What we built

- **Upstream contribution** — a real PR-ready branch on my fork `nissimbrami/GPTCache`, ready to open against `zilliztech/GPTCache`. 3 files, +389 lines.
- **Research paper** — an 8-page ACM `acmart` sigconf write-up in `task2-final-project/report-latex/repo`
- **Plugin code** — `code/src/cost_aware_eviction/`, 1,074 lines. Heart is `eviction_manager.py` — an indexed min-heap with $O(\log n)$ priority updates, thread-safe via a single `Lock`.
- **Test suite** — **304 tests**, all passing. Cover factory wiring, backwards-compat API, cost-aware ranking, size penalisation, clock monotonicity, tie-breaking, input validation.
- **Benchmark harness** — 6 workload generators, a runner, metrics.
- **Statistics script** — paired-t + Bonferroni + BCa bootstrap analysis.

4.5 The API design

Backwards compatible while adding new capability:

- `EvictionBase("memory", policy="GDSF")` — factory dispatches to the new class. Existing users unaffected.
- `put(keys)` — still works. Defaults cost = 1, size = 1, which reduces GDSF to LFU-with-clock-aging.
- `put_with_metadata([(key, cost, size), ...])` — the new extension. Unlocks the full cost-aware behaviour.

4.6 The benchmark

3,600 runs: 30 seeds × 6 workloads × 4 cache sizes × 5 policies. Every run reports **CWHR** (cost-weighted hit rate) and **dollar spend**. Then paired-t + Bonferroni + 95% BCa bootstrap CIs (10,000 resamples, seed pinned to `20260721`).

4.7 The results

Workload	Δ CWHR	95% BCa CI	Bonf. p	Dollar Δ
high-variance cost	**+0.1190**	[+0.102, +0.136]	8.6e-26	**+25.7%**
bursty	**+0.1626**	[+0.150, +0.175]	5.9e-49	**+32.3%**
size-varying	**+0.1632**	[+0.153, +0.173]	1.6e-58	**+91.0%**
adversarial anti-LRU	**+0.1419**	[+0.100, +0.189]	3.4e-8	**+18.8%**
uniform cost	+0.00003	[−0.0004, +0.0005]	1.00	+0.005%
Zipf, fits in cache	−0.0003	[−0.0005, −0.0002]	4.9e-4	−0.037%

Read it this way: on the four workloads where a cost-aware policy *should* help, GDSF wins by **18.8%–91.0% in dollar savings** vs LRU, with Bonferroni-corrected p ranging from 10^{-8} to 10^{-58}. On the two workloads where cost-awareness cannot help, GDSF is statistically indistinguishable from LRU — no meaningful regression.

4.8 The elevator pitch

> *I ported GDSF — a classic weighted-caching policy from the 1990s for web proxies — into GPTCache, the ∼12,000-star Zilliz project that puts a semantic response cache in front of LLM APIs. On realistic LLM workloads where regeneration cost varies by 100× between entries, my policy saves up to **91%** more money than the default LRU with $p < 10^{-58}$. Contribution is a PR-ready branch on `zilliztech/GPTCache`; the write-up is an 8-page ACM sigconf paper.*

Cross-reference: how Task 1 and Task 2 relate

Two caches, one idea:

Layer	Task 1 — StreamingLLM	Task 2 — GDSF on GPTCache
What is cached	Key/Value vectors of past tokens	Whole (prompt → response) pairs
Where in the stack	Inside the model, per attention layer	In front of the model, at the API layer
Capacity unit	S sinks + L rolling tokens	N entries or M bytes
Eviction signal	Position (first S + last L)	Frequency × cost / size + clock
Failure mode when dumb	PPL 5158 (softmax collapse)	Overspend on regeneration $$$
Fix	Positional eviction rule	Cost-aware eviction rule

Both caches are bounded. Both suffer under naive eviction. Both admit a small, elegant policy fix that doesn't require retraining the model. That is the through-line for the whole project and it is what the last slide of the presentation says.